

ТЗ: учебная БД «Университет»

Цель: спроектировать и написать SQL-скрипт создания БД самостоятельно, отработав PK, UNIQUE, CHECK и связи 1:1, 1:N, M:N.

Ниже — только требования. Схему, названия столбцов, типы данных и сам SQL нужно придумать и написать самому.

1. Сущность «Кафедра»

Хранит кафедры университета.

Требования:

- У кафедры есть уникальный идентификатор.
 - Название кафедры не должно повторяться.
 - У кафедры есть бюджет, который не может быть нулевым или отрицательным.
-

2. Сущность «Преподаватель»

Хранит преподавателей.

Требования:

- У преподавателя есть уникальный идентификатор.
 - Email преподавателя уникален во всей таблице.
 - У преподавателя есть ставка (доля занятости), допустимы только значения: 0.25, 0.5, 0.75, 1.0 — любое другое значение должно быть запрещено на уровне БД.
 - Каждый преподаватель обязательно относится ровно к одной кафедре.
 - Одна кафедра может иметь много преподавателей (связь **1 ко многим**).
 - Если попытаться удалить кафедру, у которой ещё остались преподаватели — подумайте, как должна вести себя БД (это тоже часть задания: выбрать и обосновать поведение внешнего ключа).
-

3. Сущность «Пропуск»

Хранит пропуска (бейджи) преподавателей.

Требования:

- У пропуска есть уникальный идентификатор.
- Номер пропуска уникален.
- У пропуска есть дата выдачи.
- У каждого преподавателя может быть **не более одного** пропуска, и один пропуск принадлежит ровно одному преподавателю (связь **один к одному**).

- Подумайте, как на уровне ограничений БД (а не только в приложении) гарантировать, что один и тот же преподаватель не сможет получить два пропуска.
-

4. Сущность «Студент»

Требования:

- У студента есть уникальный идентификатор.
 - Email студента уникален.
 - Возраст студента должен находиться в диапазоне от 16 до 80 лет включительно — значения вне диапазона должны блокироваться самой БД.
-

5. Сущность «Курс»

Требования:

- У курса есть уникальный идентификатор.
 - Название курса уникально.
 - Количество часов курса должно быть строго больше нуля.
-

6. Связь «Студент — Курс» (записи на курс)

Требования:

- Один студент может быть записан на много курсов, и один курс может содержать много студентов (связь **многие ко многим**) — то есть напрямую связать таблицы «Студент» и «Курс» нельзя, нужна дополнительная сущность.
 - В этой связующей сущности нужно хранить оценку студента по курсу.
 - Оценка может отсутствовать (курс ещё не сдан), но если она указана — должна быть в диапазоне от 0 до 100.
 - Один и тот же студент не может быть записан на один и тот же курс дважды — это должно быть невозможно физически на уровне ключа таблицы, а не проверяться в коде приложения.
-

Что нужно сдать (написать самому)

1. SQL-скрипт создания всех таблиц ((CREATE TABLE)) со всеми ограничениями из ТЗ.
2. Скрипт с тестовыми (INSERT)-данными (минимум по 2 записи в каждую таблицу, с осмысленными связями между ними).
3. Минимум 4 (INSERT)-запроса, которые нарушают одно из требований выше (по одному на PK/UNIQUE/CHECK/связь) — заготовленные, но закомментированные, чтобы можно было раскомментировать и увидеть ошибку.
4. Три (SELECT)-запроса с (JOIN):
 - вывести преподавателей вместе с названием их кафедры;

- вывести преподавателей вместе с номером их пропуска;
- вывести студентов вместе с названиями курсов, на которые они записаны, и их оценками.

Чек-лист самопроверки

- ☐ В каждой таблице есть первичный ключ (в одной из таблиц — составной).
- ☐ Использовано хотя бы 4 ограничения `UNIQUE`.
- ☐ Использовано хотя бы 4 ограничения `CHECK`.
- ☐ Реализована связь 1:1 (через `UNIQUE` на внешнем ключе).
- ☐ Реализована связь 1:N (обычный внешний ключ).
- ☐ Реализована связь M:N (через отдельную связующую таблицу с составным РК).
- ☐ Каждое ограничение проверено вручную — вы попытались его нарушить и увидели ошибку СУБД.